

Bases de données ING2

ECE-Paris

Contenu

1. Modélisation BDD
2. Normalisation BDD
3. Le langage SQL

Répartition des Séances

Semaine	TD/TP
09/02/2026	Modélisation (1h de préparation +3h TD/TP1)
16/02/2026	Modélisation (2h TD/TP1) + Normalisation (1h de préparation + 1h TD/TP2)
23/02/2026	Normalisation(2h TD/TP2) + SQL (1h de préparation + 1hTD/TP4)
09/03/2026	SQL(3h TD/TP4) + TEST 1 (1h sur modélisation et normalisation)
16/03/2026	SQL(4h TD/TP5)
23/03/2026	SQL(3h TD/TP6) + TEST 2 (1h sur SQL)
30/03/2026	DS (1,5h tout le programme)

Contenu

1. Modélisation BDD

2. Normalisation BDD

3. Le langage SQL

Introduction

Dans la vie de tous les jours

RATP : transports à Paris et en Île-de-France

https://www.ratp.fr

Rechercher

Où allons-nous ?

De lieu, station, adresse

À lieu, station, adresse

Itinéraires accessibles

Départ Arrivée

15/11/2019 à 17 15

C'est parti

Métro, bus, tramway, consultez les plans

Boutique RATP

RATP Group

FRANÇAIS

Se connecter à maRATP

ITINÉRAIRES HORAIRES PLANS VISITER PARIS

En direct du groupe RATP

Information trafic à 17:10

Meilleurs services lettriques, nouveaux prix

Service et tran et

5

5 février 2026

17:11 15/11/2019

Introduction

Pourquoi stocker des données ?

- Analyse de contenu
- Analyse décisionnelle
- Analyse prédictive



<https://www.conix.fr/business-intelligence-reporting-a-lanalyse-predictive/>

<https://itsocial.fr/enjeux-it/enjeux-innovation/intelligence-artificielle/ia-5-etapes-mettre-place-analyse-predictive/>

Introduction

Les bases de données sont au cœur des Systèmes d'Information (SI) d'entreprise : bilan d'activité, fiches de paie, catalogue...

- Il existe plusieurs de types de BD
 - hiérarchiques, réseau, **relationnelles**, web,...
- Interrogeable et modifiable par un **Système de Gestion de Bases de données (SGBD)**

Sommaire

– Introduction

– **Modèle entité association**

Modèle Conceptuel des Données (MCD)

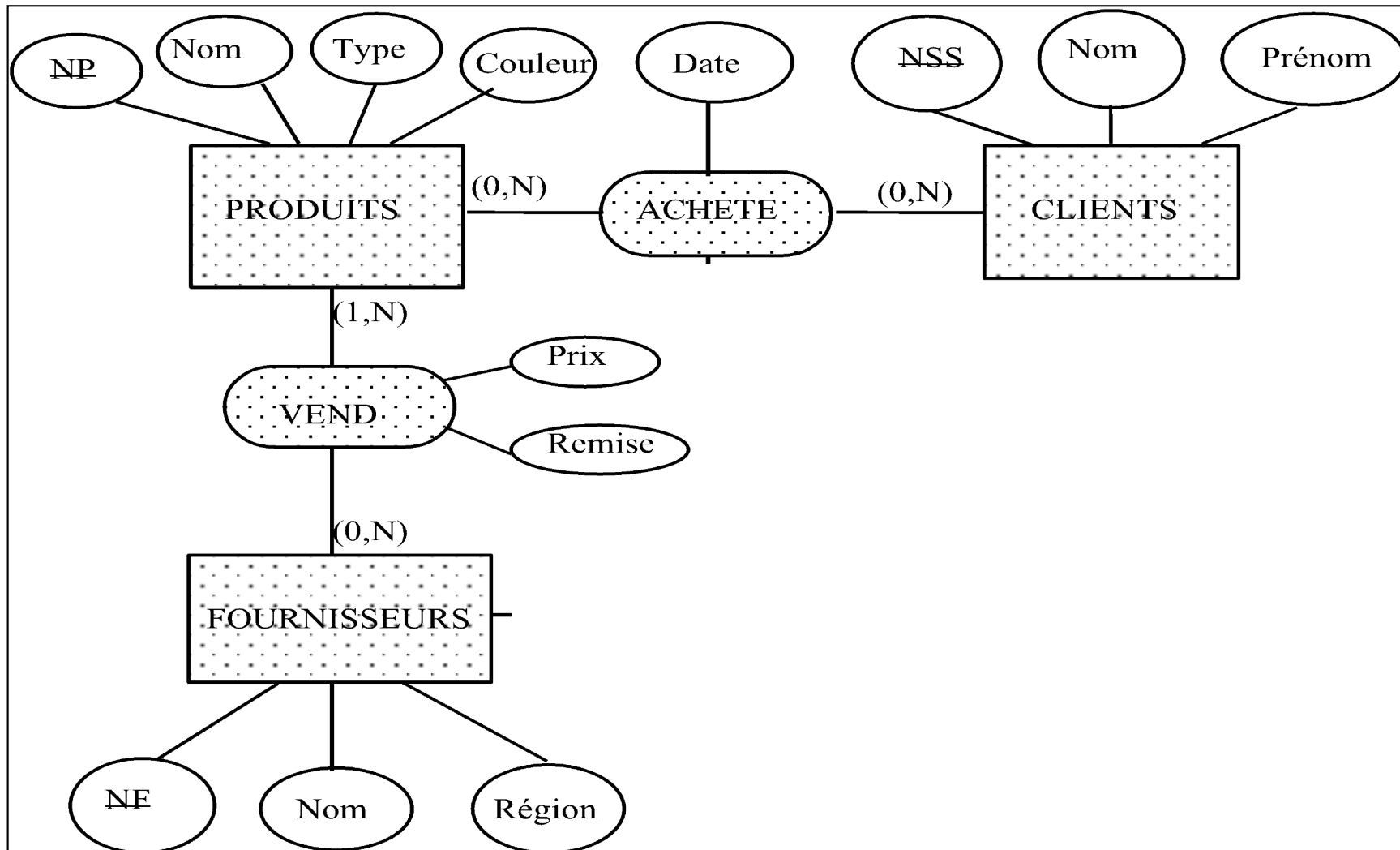
Modèle Entité – Association (EA)

Modèle Entité – Association (EA)

- Ensemble de concepts pour modéliser les données d'une application
- Ensemble de symboles graphiques associés
- Formalisé en 1976 par P. Chen
- Etendu vers E/R généralisé puis vers l'objet

Modèle Entité – Association (EA)

Exemple de modèle EA

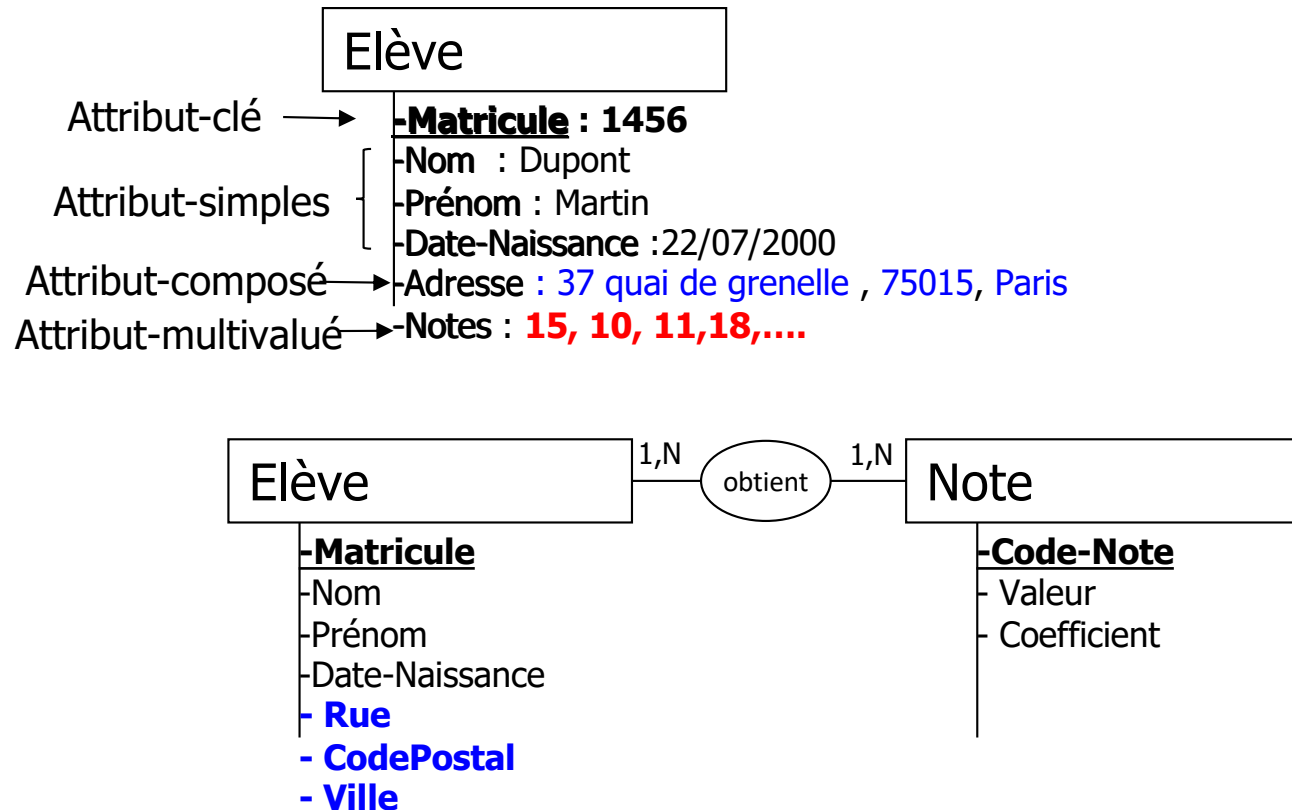


Modèle Entité – Association (EA)

- **Entité** : Un objet du monde réel qui peut être identifié et que l'on souhaite représenter
- **Attribut** : Description des propriétés des entités
 - Attribut simple: attribut ayant une valeur d'un type de base
 - Attribut composé: attribut constitué d'un groupe d'attributs
 - Attribut multi-valué: attribut pouvant avoir plus d'une valeur
- **Identifiant ou Clé**: identifier d'une façon unique l'entité
 - Un identifiant peut être constitué de plusieurs attributs (clé composée)
- **Association** : Les entités sont reliées ensemble par des associations
 - Une association peut avoir des attributs (propriétés)
 - Elle peut relier plusieurs entités ensemble
 - Il est possible de distinguer le rôle d'une entité (elle peut en avoir plusieurs)
- **Cardinalité (Multiplicity)**
 - Le nombre d'instance d'une entité pour chaque instance de l'autre

Modèle Entité – Association (EA)

Exemple



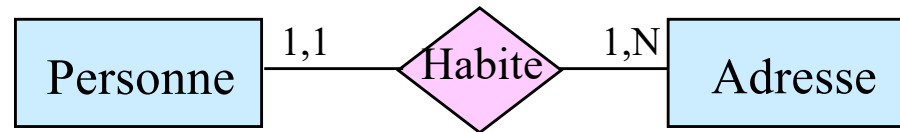
Modèle Entité – Association (EA)

Cardinalité d'une association

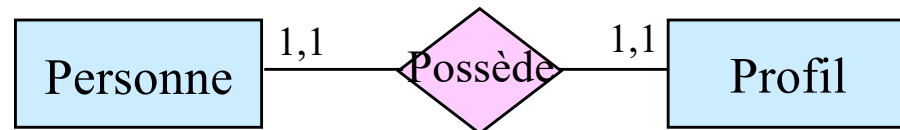
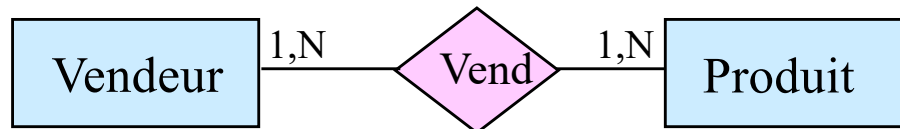
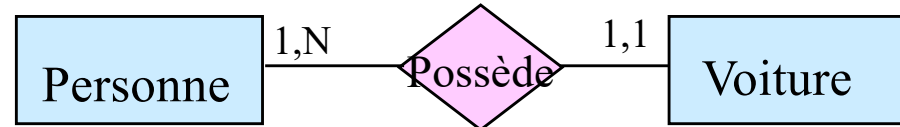
Elle se note **(mini, maxi)** sur chaque "branche" de l'association.

Mini : au moins

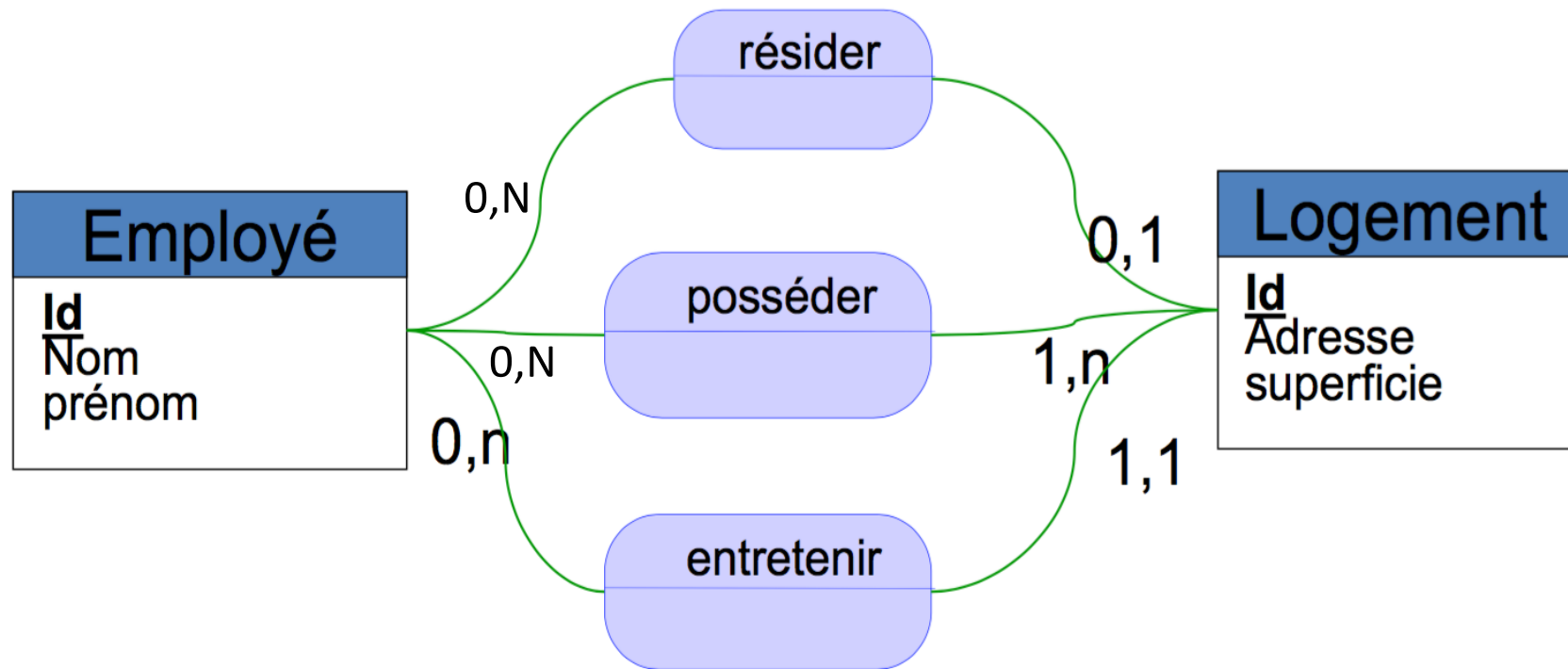
Maxi : au plus



(Cardinalité
maximum,
Cardinalité
minimum) :
(0, N), (1,N), (0,1),
(1,1)



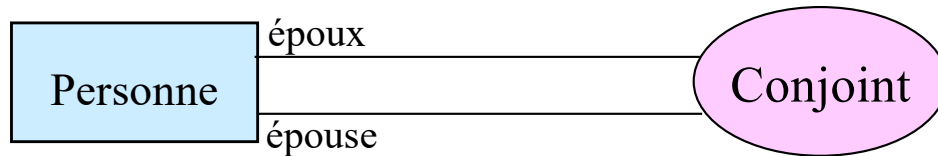
Modèle Entité – Association (EA)



Un employé peut être propriétaire, locataire et/ou Chargé de l'entretien d'un logement

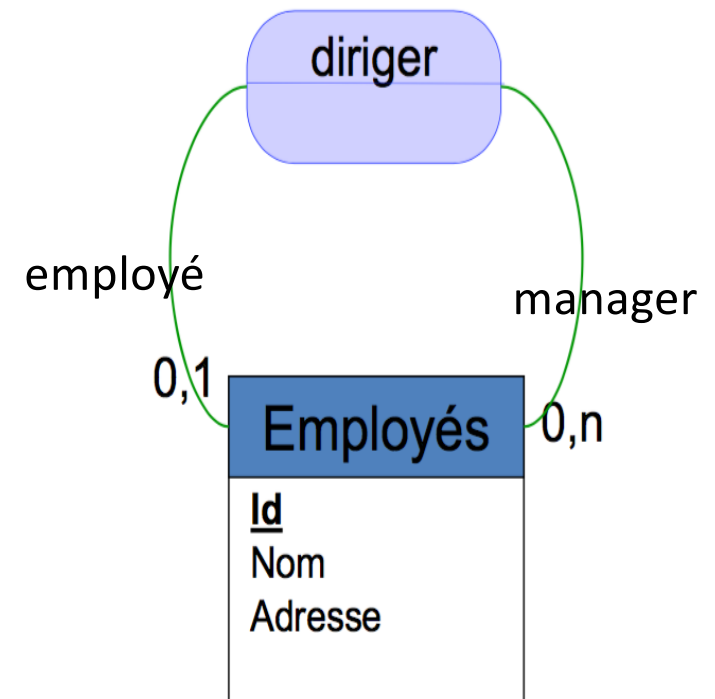
Modèle Entité – Association (EA)

Association réflexive



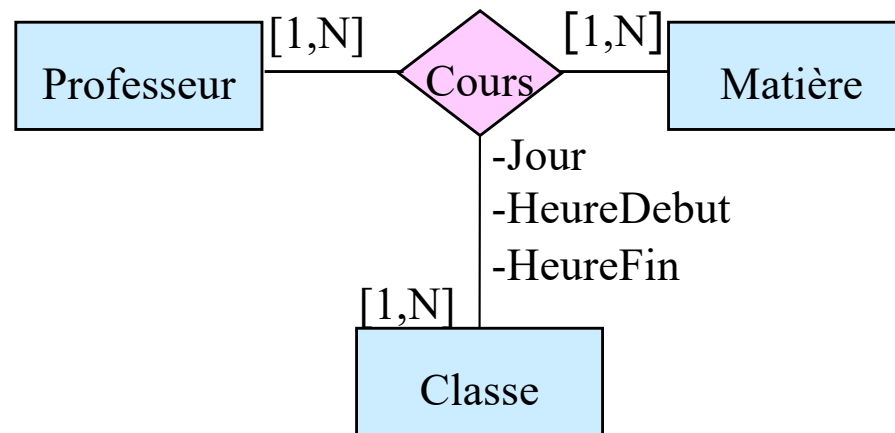
Remarque : on ajoute souvent des noms de rôles pour distinguer les deux occurrences

Un employé est dirigé par un autre employé (sauf le directeur général) et un employé peut diriger plusieurs employés



Modèle Entité – Association (EA)

Association n-aire



Exercice d'application

Enoncé

Un client a un numéro de sécurité sociale, un nom, un prénom, un bonus et un malus. Il passe un contrat pour chacun de ses véhicules, Ce contrat est géré par agent et il couvre certains risques et concerne un véhicule. Un véhicule est caractérisé par un numéro, une puissance, une marque, un type et une couleur. Les clients ont parfois des sinistres avec des tierces personnes. Les tierces personnes ont un numéro de sécurité sociale et sont assurées auprès d'une compagnie d'assurance. Une compagnie d'assurance a un nom et une adresse. On doit connaître le lieu et la date du sinistre

Question

- Elaborer le schéma Entité/Association correspondant à la description d'une compagnie d'assurance donnée ci-dessus

Exercice d'application

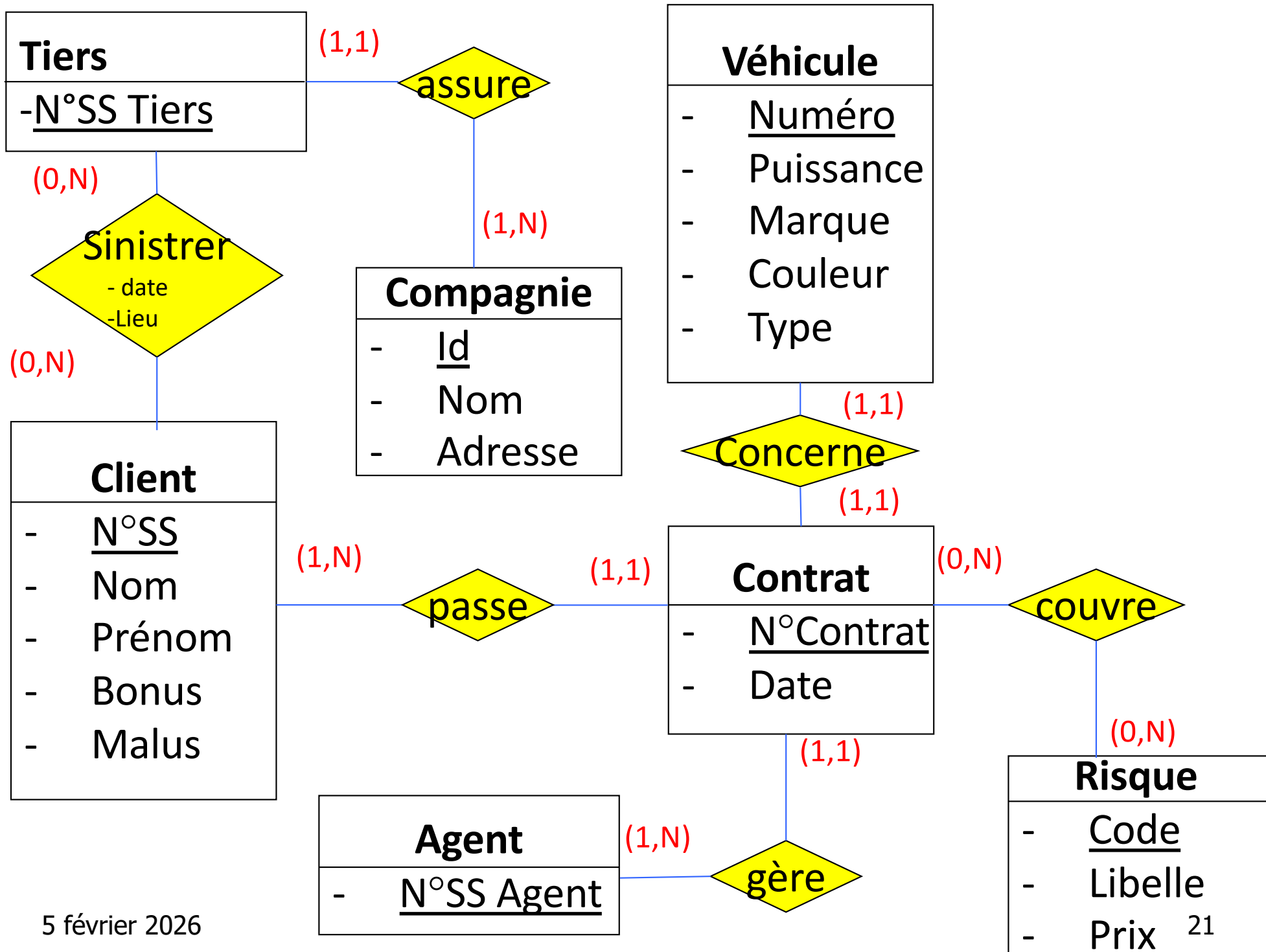
Enoncé

Un client a un numéro de sécurité sociale, un nom, un prénom, un bonus et un malus. Il passe **un contrat** pour chacun de ses véhicules, Ce contrat est géré par **agent** et il couvre certains **risques** et concerne **un véhicule**. Un véhicule est caractérisé par un numéro, une puissance, une marque, un type et une couleur. Les clients ont parfois des sinistres avec **des tierces personnes**. Les tierces personnes ont un numéro de sécurité sociale et sont assurées auprès d'une **compagnie d'assurance**. Une compagnie d'assurance a un nom et une adresse. On doit connaître le lieu et la date du sinistre

Exercice d'application

Enoncé

Un client a un numéro de sécurité sociale, un nom, un prénom, un bonus et un malus. Il **passé un contrat** pour chacun de ses véhicules, Ce contrat est **géré** par **agent** et il **couvre** certains **risques** et **concerne un véhicule**. Un véhicule est caractérisé par un numéro, une puissance, une marque, un type et une couleur. Parfois les clients **ont des sinistres** avec **des tierces personnes**. Les tierces personnes ont un numéro de sécurité sociale et sont **assurées** auprès d'une **compagnie d'assurance**. Une compagnie d'assurance a un nom et une adresse. On doit connaître le lieu et la date du sinistre



Sommaire

- Introduction
- Modèle entité association
- **Modèle relationnel**

Modèle Logique de Données (MLD)

Le modèle relationnel (MR)

Modèle Relationnel (MR)

Ce modèle est proposé par Edgar Frank CODD (Chercheur chez IBM) à la fin des années 1960

Il proposait de stocker des données hétérogènes dans des tables, permettant d'établir des relations entre elles

Modèle Relationnel (MR)

Le passage au relationnel

- Implémentation des entités et associations sous forme de tables
- Les attributs correspondent aux colonnes des tables
 - nom attribut → nom colonne
 - Ensemble de valeurs → domaine

Relation ou table

Champs, attributs, colonnes,
propriétés

Id-D	Nom	Prénom
1	Dupont	Pierre
2	Durand	Paul
3	Masse	Jean
....		

Tuples, lignes ou n-uplets

Modèle Relationnel (MR)

Remarques :

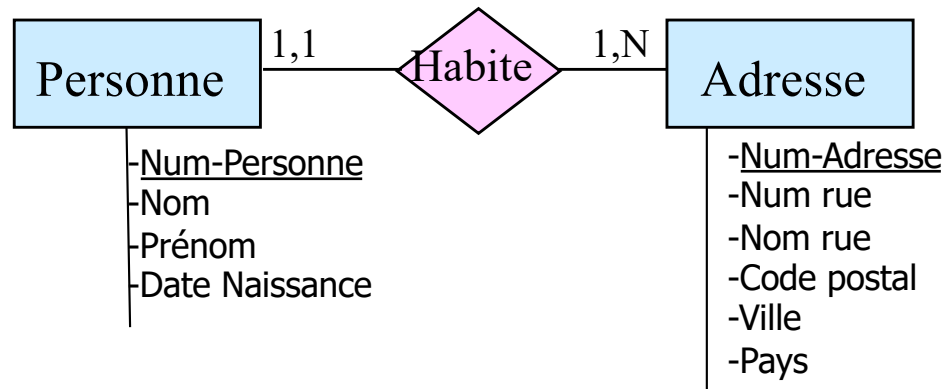
- Le nombre d'attributs est appelé : *degré*
- Le nombre de tuples est appelé : *cardinalité*
- Exemple :

Degré = 3

Cardinalité = 2

Niveau	Nom	Prénom
1A	Bernard	Jean
2A	Dupont	Eric

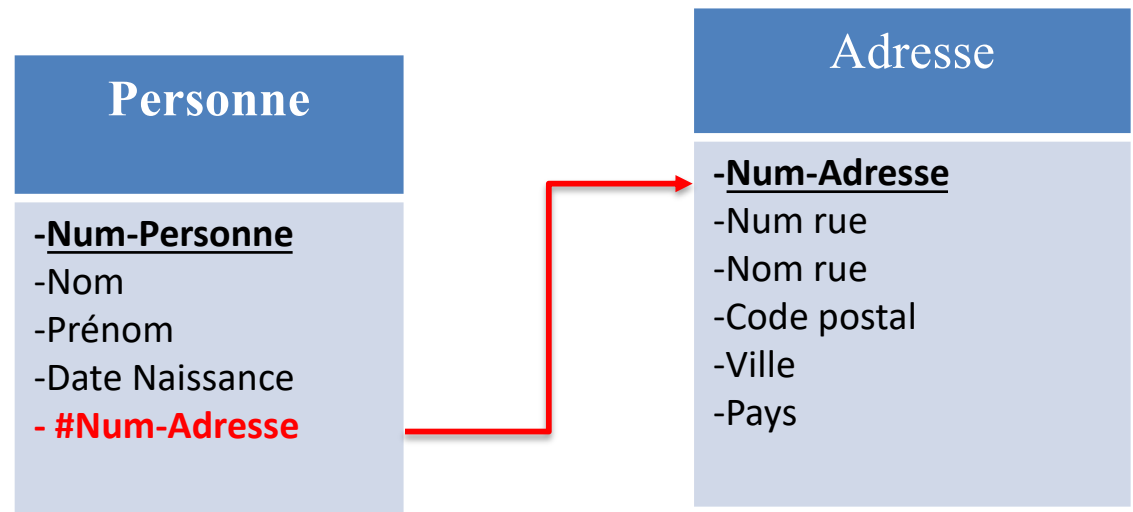
Modèle Relationnel (MR)



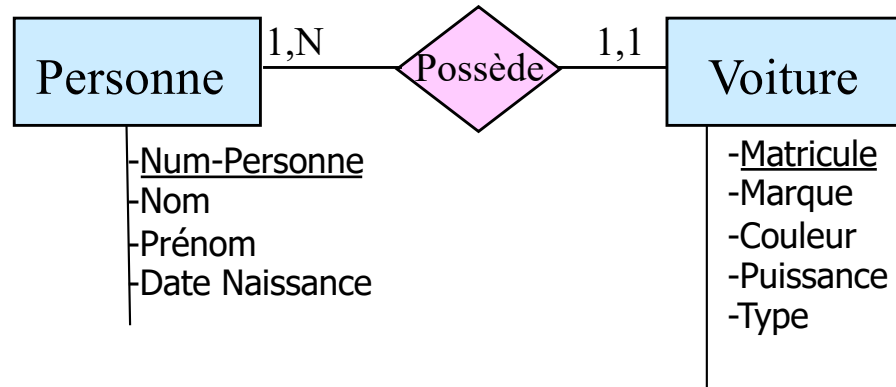
Les tables/relations :

Personne (Num-Personne, Nom, Prénom, Date naissance, #Num-Adresse)

Adresse (Num-Adresse, Num rue, Nom rue, Code postal, Ville, Pays)



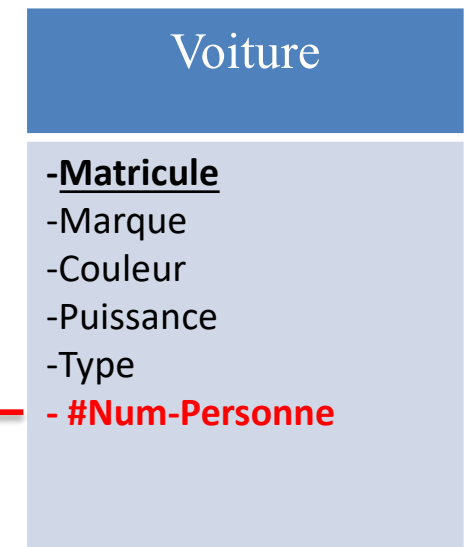
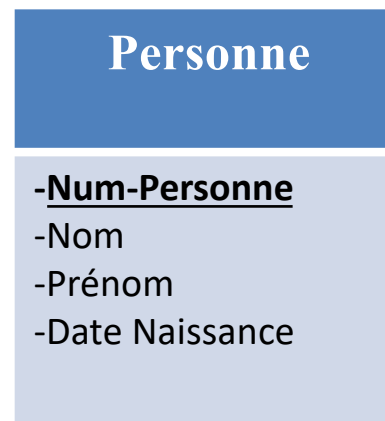
Modèle Relationnel (MR)



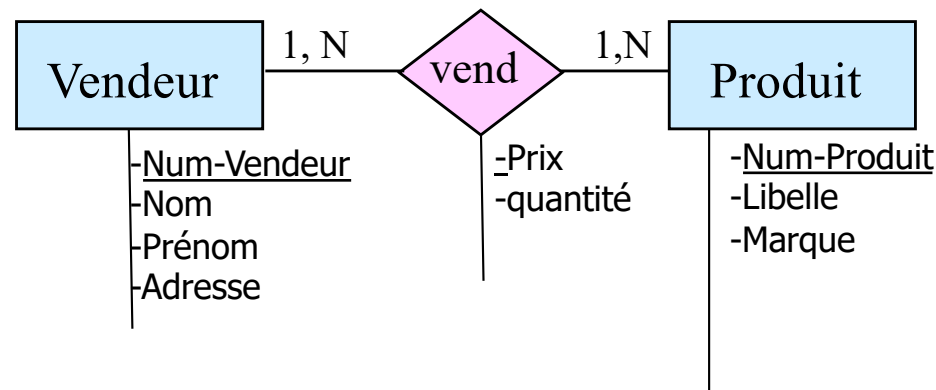
Les tables/reliations

Personne (Num-Personne,
Nom, Prénom, Date naissance)

Voiture (Matricule, Marque,
Couleur, Puissance, Type,
#Num-Personne)



Modèle Relationnel (MR)



Les tables/relations :

Vendeur (Num-Vendeur, Nom, Prénom, Adresse)

Produit (Num-Produit, Libelle, Marque)

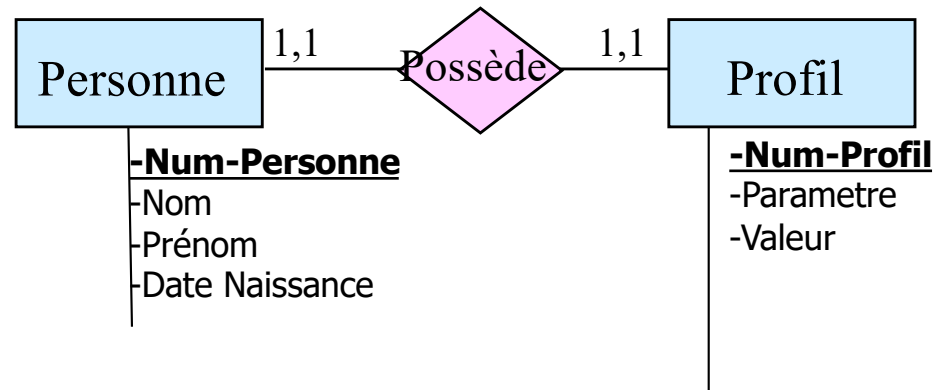
Vente (#Num-Vendeur, # Num-Produit, Prix, Quantité)

OU

Vente (Num-Vente, Prix, Quantité, #Num-Vendeur, # Num-Produit)



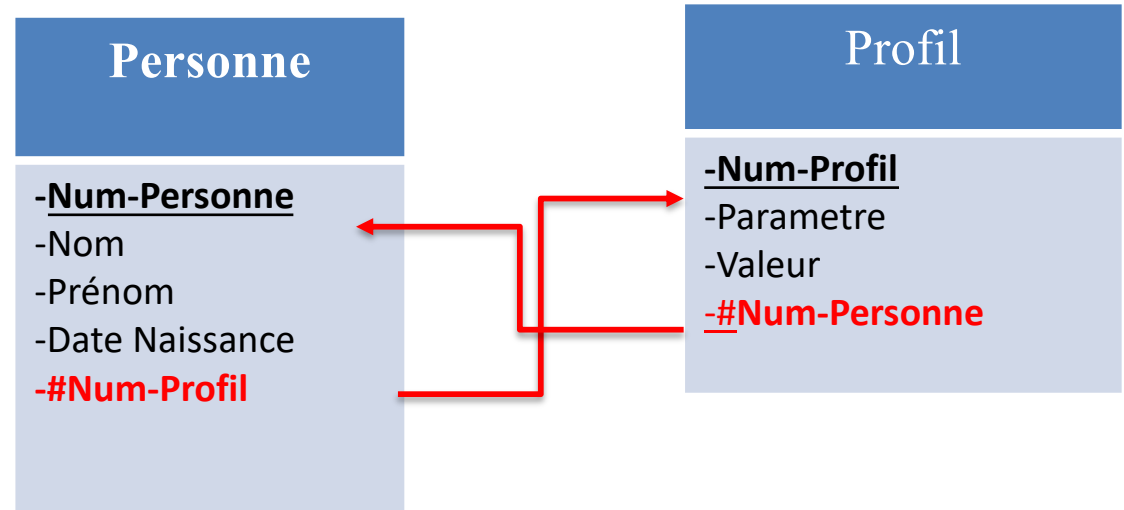
Modèle Relationnel (MR)



Les tables/relations :

Personne (Num-Personne, Nom, Prénom, Date naissance, #Num-Profil)

Profil (Num-Profil, paramètre, valeur, #Num-Personne)



Exercice d'application

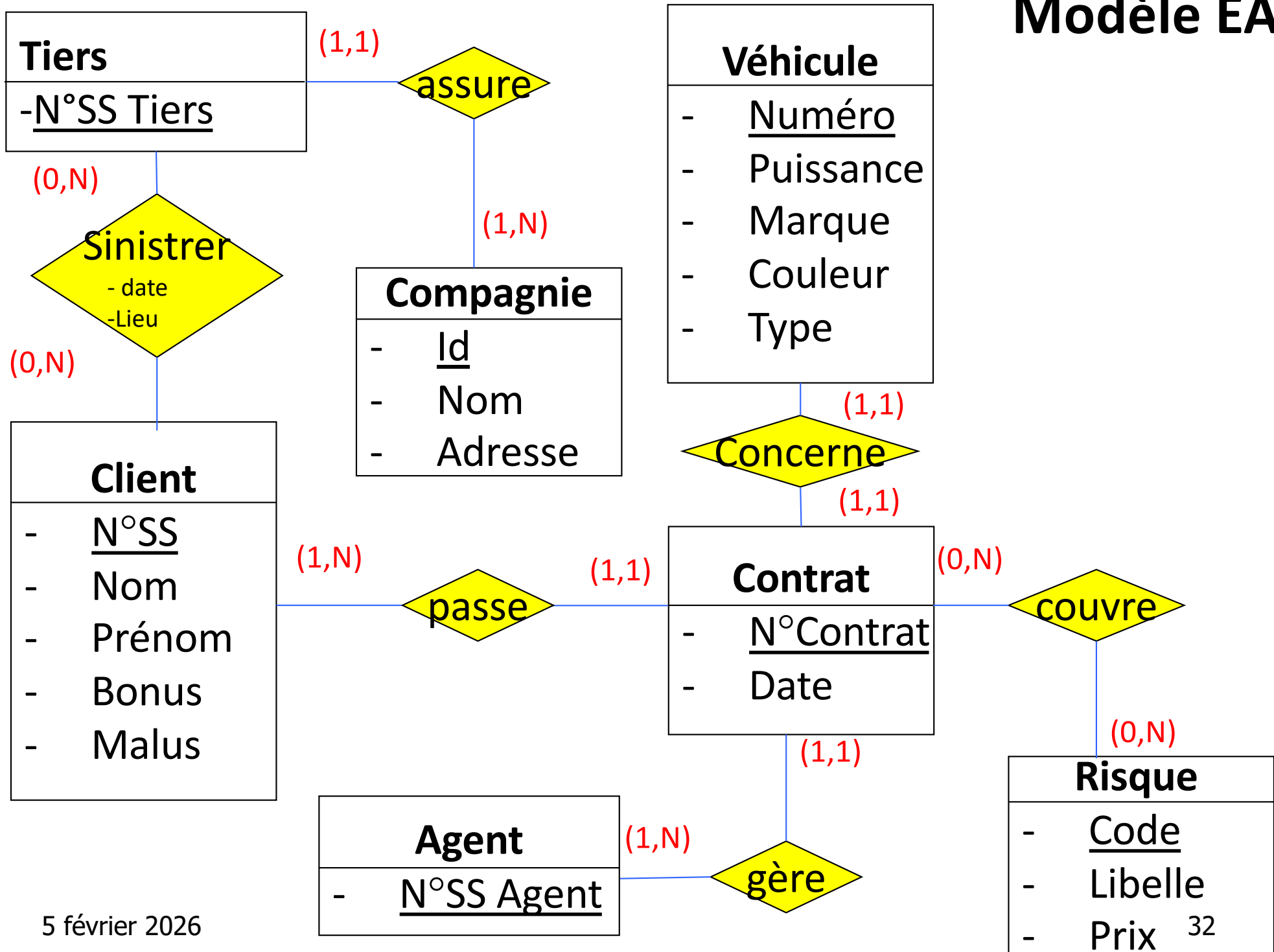
Enoncé

Un client a un numéro de sécurité sociale, un nom, un prénom, un bonus et un malus. Il **passé un contrat** pour chacun de ses véhicules, Ce contrat est **géré** par **agent** et il **couvre** certains **risques** et **concerne un véhicule**. Un véhicule est caractérisé par un numéro, une puissance, une marque, un type et une couleur. Parfois les clients **ont des sinistres** avec **des tierces personnes**. Les tierces personnes ont un numéro de sécurité sociale et sont **assurées** auprès d'une **compagnie d'assurance**. Une compagnie d'assurance a un nom et une adresse. On doit connaître le lieu et la date du sinistre

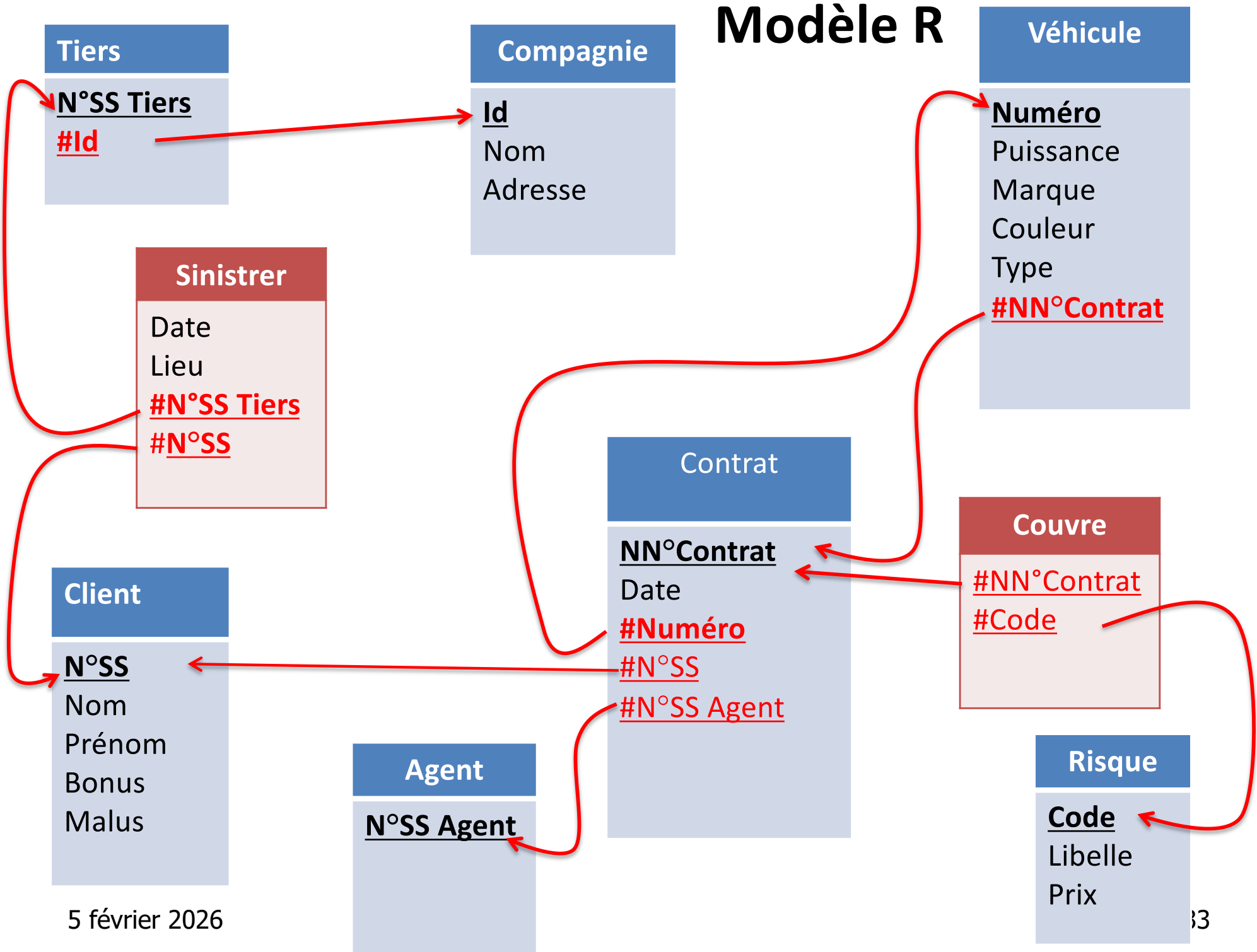
Modèle Entité-Association

Modèle Relationnel ?

Modèle EA



Modèle R



Modèle Relationnel (MR)

Modèle Relationnel

Les tables/relations :

- Client (N°SS, Nom, Prénom, Bonus, Malus)
- Contrat (N°Contrat, Date, #N°SS, #Numéro, #N°SS-Agent)
- Agent (N°SS-Agent, Nom, Prénom, Adresse,...)
- Véhicule (Numéro, Puissance, Marque, Couleur, Type, # N°Contrat)
- Risque (Code, libellé, Prix)
- Tiers (N°SS-Tiers, #Id)
- Compagnie (Id, Nom, Adresse)
- Sinistre (# N°SS, # N°SS-Tiers, date, lieu)
- Couvre_ (# N°Contrat, # Code)

Sommaire

- Introduction
- Modèle entité association
- Modèle relationnel (physique)
- **Normalisation**
- Le langage SQL

Normalisation

Dépendances Fonctionnelles

- Les DF entre les attributs d'une entité doivent vérifier la règle suivante :
 - tous les attributs d'une entité dépendent fonctionnellement et uniquement de l'identifiant
- soit $R(A_1, A_2 \dots A_n)$ un schéma de relation, X et Y des sous-ensembles de $A_1, A_2 \dots A_n$
- on dit que $X \rightarrow Y$ (X détermine Y ou Y dépend fonctionnellement de X) ssi il existe une fonction qui à partir de toute valeur de X détermine une valeur unique de Y

Normalisation

Exemples

- PERSONNE
 - N° SS → NOM ?
On instancie pour vérifier
 - 1450578.. → Dupont
 - NOM → N° SS ?
 - Dupont → 1450578.. , 262119...,

Normalisation

Exemples suite

- VOITURE

- (MARQUE, TYPE) → PUISSANCE ?

OUI

Exp: (clio3, essence 1.2L 16V) détermine 75 Ch

- MARQUE → PUISSANCE ?

NON

- PUISSANCE → TYPE ?

NON

- POSSEDE

- N° VEHP → N° PROP ?

OUI

- N° PROP → N° VEHP ?

NON

- (N° VEHP, N° PROP) → DATE ACHAT ?

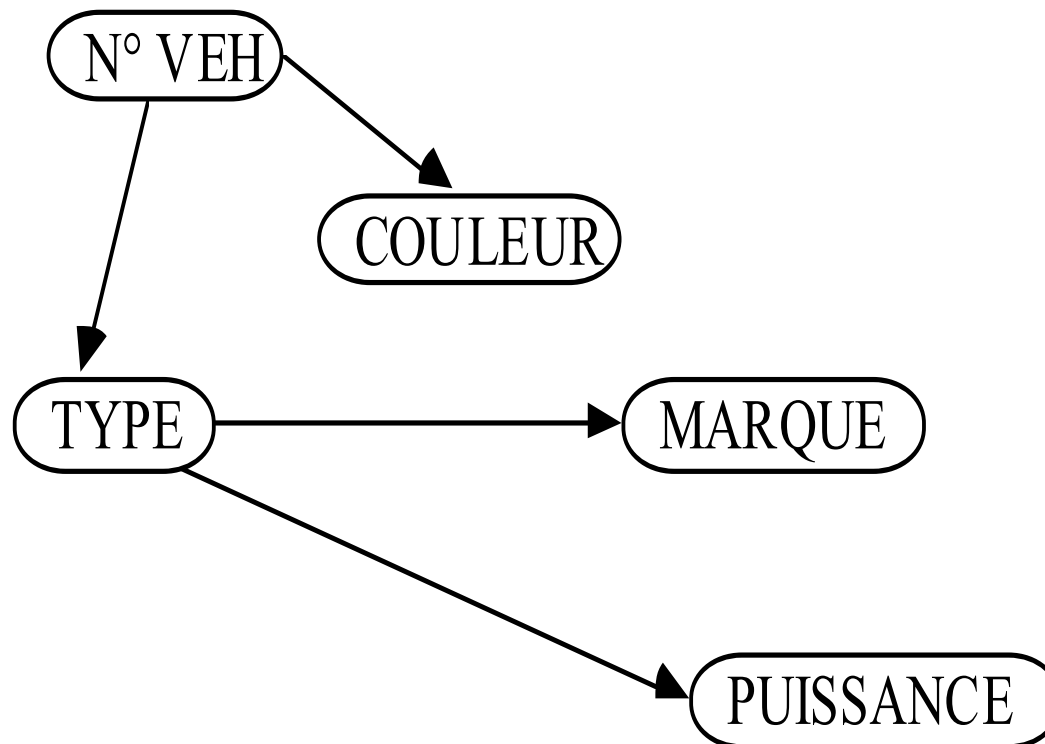
OUI

Normalisation

Exemple de graphe de DF

VOITURE1 (N°VEH, TYPE, COULEUR, MARQUE, PUISSANCE)

N° VEH → TYPE
N° VEH → COULEUR
TYPE → MARQUE
TYPE → PUISSANCE

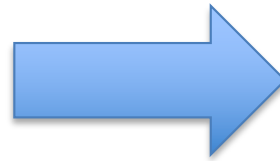


Normalisation

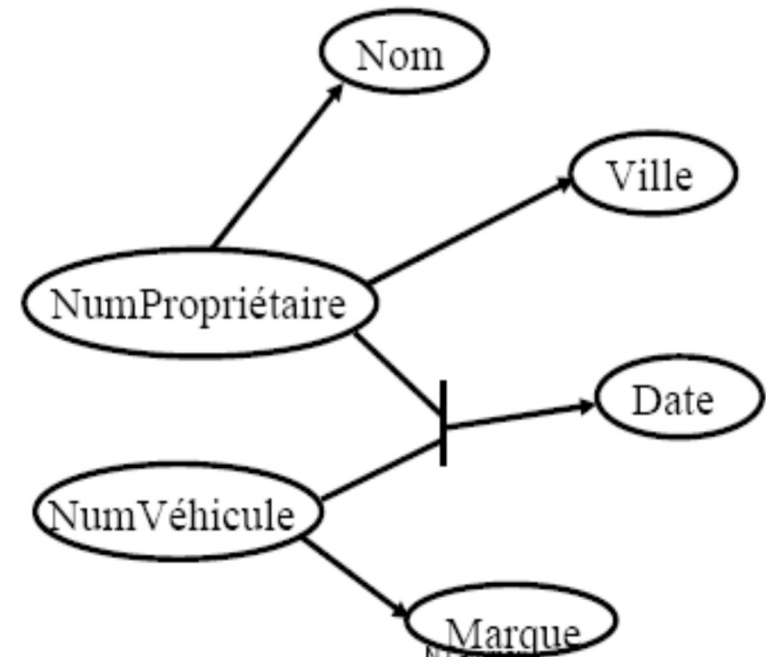
RELATION (NumPropriétaire, Nom, Ville, Date, NumVéhicule, Marque)

Ensemble des DF

- {
- ◆ NumPropriétaire → Nom
- ◆ NumPropriétaire → Ville
- ◆ NumVéhicule → Marque
- ◆ NumPropriétaire, NumVéhicule → Date
- }



Graphe de DF



Normalisation

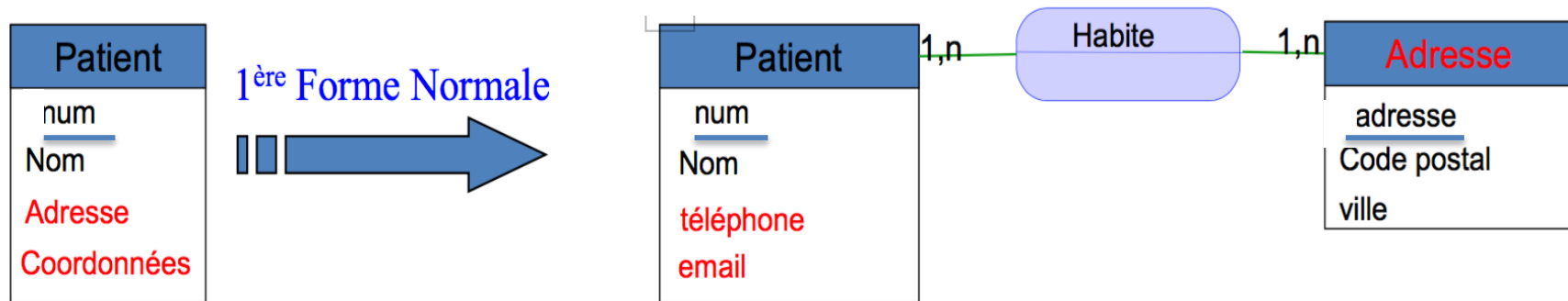
Formes normales

- **Objectifs**
 - Définir des règles pour décomposer les relations tout en préservant les DF et sans perdre d'informations, afin de représenter des objets et associations du monde réel
 - Éviter les anomalies de mises à jour
 - Éviter les réponses erronées

Normalisation

1ère Forme (1NF)

- Définition
 - Une relation est en 1NF si tout attribut contient une valeur atomique (unique)
- Exemple



Normalisation

2ème Forme (2NF)

- Définition
 - une relation est en 2NF ssi :
 - elle est en 1ère forme
 - Aucune partie de la clé ne détermine un autre attribut non clé
- Schéma



Une telle relation doit être décomposée en
 $R1(\underline{A}, \#B, D)$ et $R2(\underline{B}, C)$

Normalisation

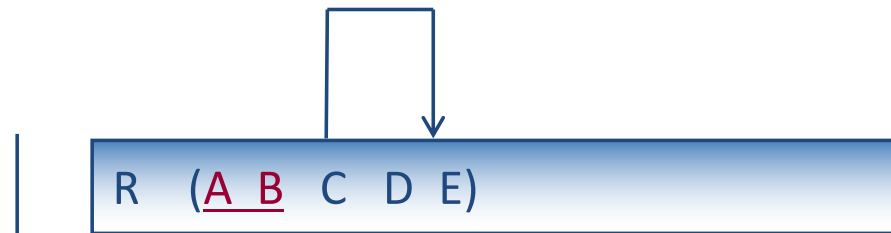
Exemple 2NF

- Fournisseur (nom, article, adresse, prix)
 - La clé est (nom, article)
 - Mais $\text{nom} \rightarrow \text{adresse}$: pas en 2NF!
- Décomposition en 2NF
 - Fournisseur (#nom, article, prix)
 - Ad-Fournisseur (nom, adresse)

Normalisation

3ème Forme (3NF)

- Définition
 - une relation est en 3NF ssi :
 - elle est en 2NF
 - Pas de dépendances entre les attributs non clés
- Schéma



Une telle relation doit être décomposée en
R1(A,B,#C,D) et R2 (C,E)

Normalisation

Exemple 3NF

- Voiture (n° veh, marque, type, puissance, couleur)
 - Type → marque
 - Type → puissance
 - Pas en 3NF !
- Décomposition en 3NF
 - Véhicule (n° veh, #type, couleur)
 - Modèle (type, marque, puissance)

Normalisation

Exemple: Normalisation par décomposition

R (NumPropriétaire, NumVéhicule, Nom, Ville, Marque, Date)

NumPropriétaire → Nom

NumPropriétaire → Ville

NumVéhicule → Marque

NumPropriétaire, NumVéhicule → Date

OUI 1 FN mais NON 2FN

R (NumPropriétaire, NumVéhicule, Nom, Ville, Marque, Date)

R1 (NumPropriétaire, NumVéhicule, Marque, Date)

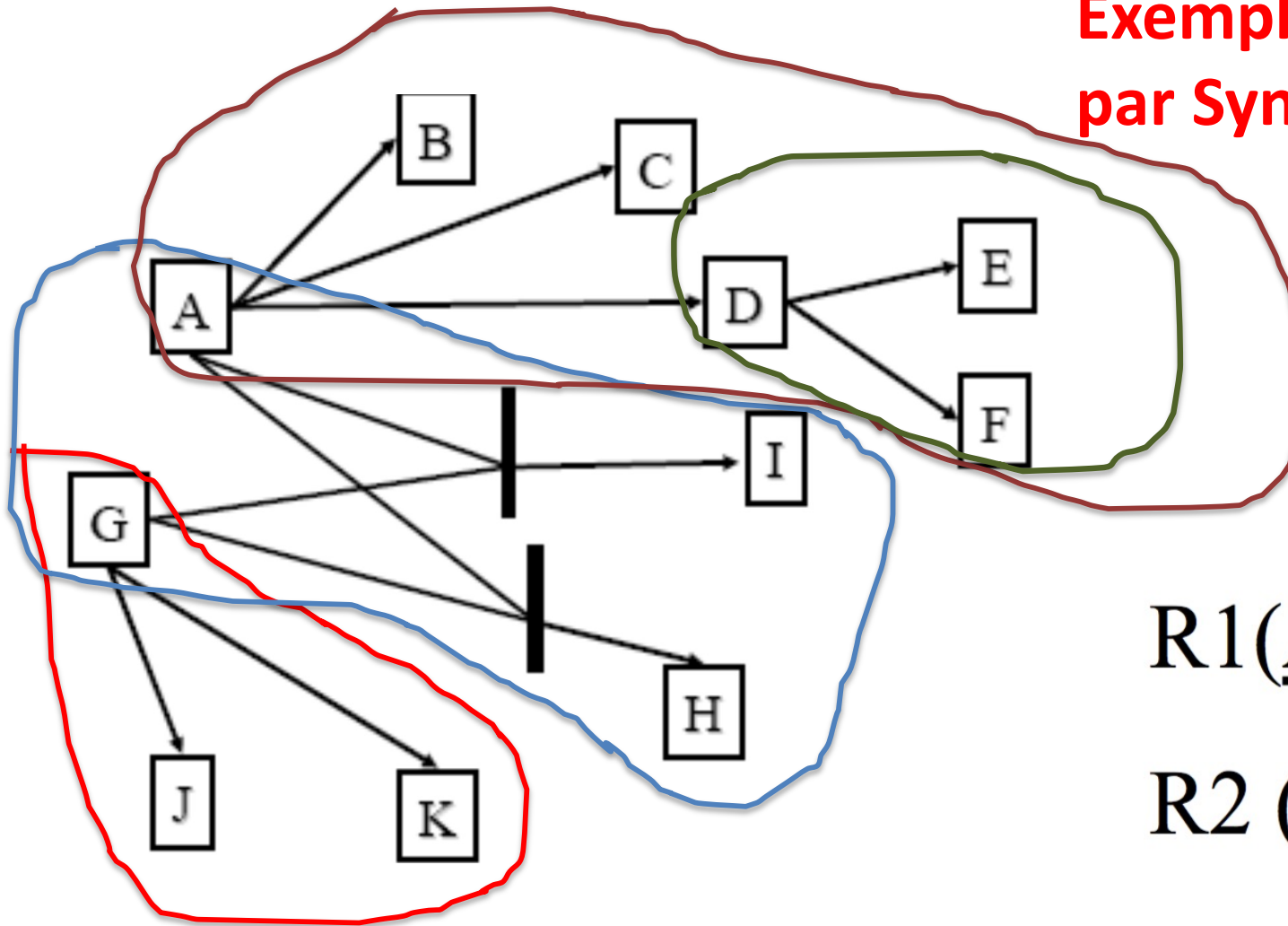
R2 (NumPropriétaire, Nom, Ville)

R11 (NumVéhicule, Marque)

R12 (NumPropriétaire, NumVéhicule, Date)

Normalisation

Exemple: Normalisation
par Synthèse



R1(A, B, C, #D)

R2 (D, E, F)

R3(#A, #G, H, I)

R4(G, J, K)

Normalisation

Propriété de la 3NF

- Toute relation R a une décomposition en relations R1, R2 ... Rn (ou plusieurs) en 3e forme normale telle que:
 - 1) pas de perte de dépendances
Les dépendances fonctionnelles des relations décomposées permettent de générer celles de la relation initiale.
 - 2) pas de perte d'informations
Les relations décomposées permettent à tout instant de recomposer la relation initiale par jointures.
- Faiblesse:
 - Il existe des relations en 3NF avec des redondances ...

Normalisation

Règles d'inférence entre le DF

- Axiomes d'Armstrong : à partir d'un ensemble F de dépendances fonctionnelles entre les attributs d'un schéma de relation R, on peut en déduire d'autres à partir des suivantes :
 - transitivité : si $X \rightarrow Y$, et $Y \rightarrow Z$, alors $X \rightarrow Z$
 - augmentation : si $X \rightarrow Y$, alors $XZ \rightarrow Y$
 - réflexivité : si X contient Y, alors $X \rightarrow Y$
 - union : si $X \rightarrow Y$ et $X \rightarrow Z$, alors $X \rightarrow YZ$,
 - pseudo-transitivité : si $X \rightarrow Y$ et $WY \rightarrow Z$, alors $WX \rightarrow Z$
 - décomposition : si $X \rightarrow Y$ et Z contenu dans Y, alors $X \rightarrow Z$

Normalisation

Couverture minimale

Exemple : Relation COURS

(NOMPROF, VILLE, DEPARTEMENT, NOMETUDIANT,
AGE, COURS, NOTE)

- | | | |
|------------------------------|---|--|
| 1. NOMPROF → VILLE | } | n'est pas minimal car 3 est redondante |
| 2. VILLE → DEPARTEMENT | | |
| 3. NOMPROF → DEPARTEMENT | | |
| 4. NOMETUDIANT → AGE | | |
| 5. NOMETUDIANT, COURS → NOTE | | |
| 6. COURS → NOMPROF | | |

{1, 2, 4, 5, 6} est une couverture minimale

{1, 3, 4, 5} n'est pas une couverture

Normalisation

Exercice

Soit R une relation dont le schéma est le suivant : **R (UtilisateurID, Nom, Prénom, AdresseEmail, Login, Passwd, ServeurMail)**

1) Exprimer à l'aide de DF les contraintes suivantes :

- On peut déduire le nom et le prénom d'un utilisateur à partir de son identificateur
- Un utilisateur (identifié par son identificateur) possède un seul login et un seul password par serveur mails
- Une adresse email est associée à un et un seul identificateur d'utilisateur (attention : un utilisateur peut posséder plusieurs adresses mails)
- Une adresse mail est associée à un et un seul serveur mails

2) Quelle est la couverture minimale de R

3) Indiquer à partir de la couverture minimale les clés de la relation R

4) Indiquer le degré de normalité de la relation R

5) Proposer une décomposition possible de R (si nécessaire).

Normalisation

Solution

1)

A) UtilisateurId $\rightarrow$ Nom, Prénom

B) UtilisateurId , ServeurMail $\rightarrow$ Login, Passwd

C) AdresseEmail $\rightarrow$ UtilisateurId

D) AdresseEmail $\rightarrow$ ServeurMail

Normalisation

Solution

2) c'est la réponse (1) car aucune des DF n'est déduite par un des Axiomes d'Armstrong (slide 55)

Remarque

A) UtilisateurId $\rightarrow$ Nom, Prénom

B) UtilisateurId , ServeurMail $\rightarrow$ Login, Passwd

C) AdresseEmail $\rightarrow$ UtilisateurId

D) AdresseEmail $\rightarrow$ ServeurMail

E) AdresseEmail $\rightarrow$ Nom, Prénom (déduite par transitivité)

Si on ajoute par exemple la DF (E) à l'ensemble des DF

alors dans ce cas ce n'est plus une couverture minimale car la (E) est déduite

Normalisation

Solution

3)

A) ~~UtilisateurId~~ → ~~Nom~~, ~~Prénom~~

B) ~~UtilisateurId~~, ~~ServeurMail~~ → ~~Login~~, ~~Passwd~~

C) AdresseEmail → ~~UtilisateurId~~

D) AdresseEmail → ~~ServeurMail~~

La clé de la relation est : **AdresseEmail**

R (UtilisateurID, Nom, Prénom, AdresseEmail, Login, Passwd, ServeurMail)

Normalisation

Solution

R (UtilisateurID, Nom, Prénom, AdresseEmail, Login, Passwd, ServeurMail)

4) **R est en 2FN** mais pas 3FN car il existe des attributs non clés qui déterminent d'autres attributs non clés

Ex: UtilisateurId $\rightarrow$ Nom, Prénom

UtilisateurId , ServeurMail $\rightarrow$ Login, Passwd

5) R1(AdresseMail, UtilisateurId, ServeurMail)

R2(UtilisateurId, Nom, Prénom)

R3(UtilisateurId , ServeurMail, Login, Passwd)

Sommaire

- Introduction
- Modèle entité association
- Modèle relationnel (physique)
- Vérification et normalisation
- **Le langage SQL**

SQL

- Une interface plus simple pour les utilisateurs
 - La majorité des manipulations utilisateurs se font via des formulaires !
 - Mais rend possible les requêtes libres (ex: analyse)
- Simplifier le développement des applications
 - Code plus compact et facile à valider/maintenir
 - Indépendance physique et logique des programmes par rapport aux données
 - Optimisation automatique et dynamique des requêtes par le SGBD
 - Une puissance d'expression exploitable de nb façons
 - Sélection, mises à jour, intégrité, droits d'accès, audit ...

SQL

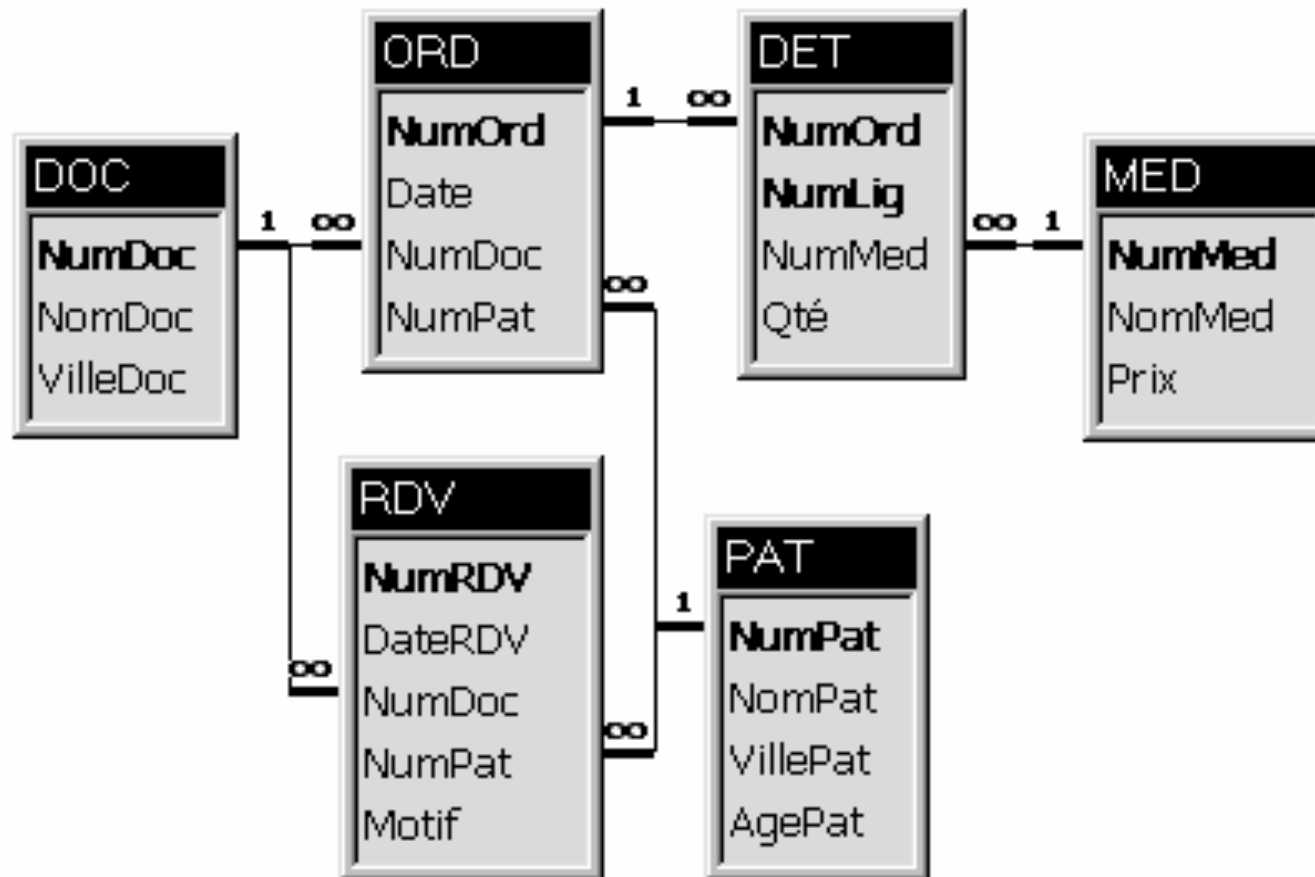
- Trois versions normalisées de SQL :
 - SQL1 (1986) : version minimale
 - SQL1 (1989) : addendum (intégrité)
 - SQL2 (1992) : version étendue à trois niveaux de conformité (entry, intermediate, full)
 - SQL3 (1999) : version étendue à la gestion d'objets complexes
 - SQL3 (2006) : extension à XML

SQL

- LDD : LANGUAGE DE DEFINITION DE DONNEES
 - CREATE TABLE
 - CREATE VIEW
 - ALTER
 - ...
- LMD : LANGUAGE DE MANIPULATION DE DONNEES
 - SELECT
 - INSERT
 - UPDATE
 - DELETE
 - ...
- *SQL procedural* : INTEGRATION AUX LANGAGES DE PROGRAMMATION
 - EXEC SQL
 - MODULE
 - PROCEDURE ..
- LCD : langage de contrôle des données
- LCT : langage de contrôle des transactions

SQL

Exemple de base de données/modèle relationnel



SQL

Création de table : Syntaxe

- **Syntaxe**

```
CREATE TABLE <relation name>  
(<attribute definition>+)  
    [{PRIMARY KEY | UNIQUE} (<attribute name>+)]
```

- **Exemple**

```
CREATE TABLE RDV (  
    NumRdv Integer PRIMARY KEY,  
    DateRDV Date,  
    NumDoc Integer,  
    NumPat Integer,  
    Motif Varchar(200),  
    FOREIGN KEY (NumDoc) REFERENCES DOC (NumDoc),  
    FOREIGN KEY (NumPat) REFERENCES PAT (NumPat)  
)
```

SQL

Insertion de données

- **Syntaxe**

INSERT INTO <relation name>

[(attribute [,attribute] ...)]

{VALUES <value spec.> [, <value spec.>] ...| <query spec.>}

- **Exemples :**

- INSERT INTO DOC VALUES (1, 'Dupont', 'Paris');
- INSERT INTO DOC (NumDoc, NomDoc, VilleDoc) VALUES (1, 'Dupont', 'Paris');
- INSERT INTO DOC (NumDoc, NomDoc) VALUES (2, 'Toto');

SQL

Suppression de données

- Syntaxe :

```
DELETE FROM <relation_name>  
[WHERE <search_condition>]
```

- Exemples :

Supprimer tous les docteurs

```
DELETE FROM DOC
```

Supprimer le docteur numéro 20

```
DELETE FROM DOC
```

```
WHERE NumDoc = 20
```

SQL

Modification de données

- SYNTAXE :

```
UPDATE <relation_name>  
SET <attribute> = value_expression  
[, <attribute> = value_expression ] ...  
[WHERE <search condition> ];
```

- EXEMPLES :

Modifier la ville du docteur dont le numéro est 20

```
UPDATE DOC  
SET VilleDoc = 'Valenton'  
WHERE NumDoc = 20
```

Augmenter de 10% le prix de tous les médicaments dont le nom contient 'Antibio'

```
UPDATE MED  
SET PRIX = PRIX*1.1  
WHERE NomMed LIKE '%Antibio%'
```

SQL

Interrogation de données

- Une requête SQL est un paraphrasage d'une expression de l'algèbre relationnelle en anglais
- Requête élémentaire

SELECT <liste des attributs projetés A_i >
FROM <liste des relations impliquées R_j >
WHERE <critères de recherche C_i >

- Sémantique du bloc

PROJECTION A_i (
 RESTRICTION C_i (
 PRODUIT (R_j)))

SQL

Structure générale d'une requête SQL

SELECT [DISTINCT/ALL] expression sur les attributs |*
FROM table [variable], table [variable], ...
[WHERE condition de sélection /sous-question]
[GROUP BY liste d'attributs]
[HAVING condition de sélection /sous-question]
[ORDER BY liste d'attributs DESC];

- Restriction :

- arithmétique (=, <, >, ≠, ≥, ≤, ...).

- ```
Select * FROM Etudiant WHERE Age >20 AND Ville = 'Paris'
```

- textuelle (LIKE)

- ```
Select * FROM Etudiant WHERE nom NOT LIKE 'C%'
```

- ```
Select * FROM Etudiant WHERE nom LIKE '%C'
```

- ```
Select * FROM Etudiant WHERE nom LIKE '%C%'
```

- sur intervalle (BETWEEN)

- ```
WHERE Age NOT BETWEEN 15 AND 25
```

- ```
WHERE Age <25 AND Age >15
```

- sur liste (IN)

- ```
WHERE Ville NOT IN ('Paris', 'Lille', 'Versailles')
```

- ```
Select NomEtu , AVG(notes)
```

- ```
From Etudiant
```

- ```
Where Ville='Paris'
```

- ```
Group by NomEtu
```

- ```
Having AVG(notes)>12
```

- ```
Order by NomEtu ASC (DESC)
```

- Possibilité de blocs imbriqués par :

- IN, EXISTS, NOT EXISTS, ALL, SOME, ANY

# SQL

## Evaluation d'une requête SQL

- La clause FROM : en faisant le produit cartésien de toutes les relations qui apparaissent dans la requête
- La clause WHERE : réalise la restriction et les jointures
- La clause GROUP BY: qui constitue les partitions
- La clause HAVING : qui sélectionne les partitions désirées
- La clause SELECT : qui constitue la projection finale
- La clause ORDER BY : qui trie les tuples du résultat final



# SQL

## Exemple de base de données

- Soit les tables/Relations suivantes :

### Tables :

- VITICULTEURS (**NVT**, NOM, PRENOM, VILLE, REGION)
- VINS (**NV**, CRU, MILLESIME, DEGRE, #NVT, PRIX)
- BUVEURS (**NB**, NOM, PRENOM, VILLE)
- ABUS (**#NV, #NB, DATE**, QTE)

# SQL

## Exemples

Q1: Donner les Crus des vins sans doubles

- SQL

- SELECT DISTINCT CRU
- FROM VINS;

# SQL

## Exemples

Q2: Noms des buveurs ayant bu des Beaujolais 97 ou 98

```
■ SELECT DISTINCT NOM
■ FROM BUVEURS B, VINS V, ABUS
■ WHERE B.NB = ABUS.NB
■ AND ABUS.NV = V.NV
■ AND CRU = 'BEAUJOLAIS'
■ AND MILLESIME IN (1997, 1998);
```

# SQL

## Exemples de)

- Q3 : Noms et prénoms des buveurs de vins dont le cru commence par B, de degré inconnu ou compris entre 11 et 13.
  - SELECT NOM, PRENOM
  - FROM BUVEURS B, VINS V, ABUS A
  - WHERE B.NB = A.NB AND A.NV = V.NV
  - AND CRU LIKE "B%"
  - AND ( (DEGRE BETWEEN 11 AND 13) OR (DEGRE IS NULL) )
- Q4 : Noms des crus bus par au moins un buveurs.
  - SELECT DISTINCT CRU
  - FROM VINS V
  - WHERE EXISTS ( SELECT \*
  - FROM BUVEURS B, ABUS A
  - WHERE B.NB = A.NB AND A.NV = V.NV )

# SQL

## Exemples

- Q5: Calculer le degré moyen pour chaque cru
  - SELECT CRU, AVG(DEGRE)
  - FROM VINS
  - GROUP BY CRU
- Q6 : Calculer le degré moyen et le degré minimum pour tous les crus de 94 dont le degré minimum est supérieur à 12
  - SELECT CRU, AVG(DEGRE), MIN(DEGRE)
  - FROM VINS
  - WHERE MILLESIME = 1994
  - GROUP BY CRU
  - HAVING MIN(DEGRE) > 12

# SQL

## Exemples

- Q8 : Obtenir les vins et numéros des vins de millésime 1977 et degré supérieur à 13 triés par ordre alphabétique croissant sur noms de cru et décroissant sur numéros de vins:

```
SELECT CRU, NV
FROM VINS
WHERE Millésime=1977 AND DEGRE>13
ORDER BY CRU ASC, NV DESC
```

# SQL

## Exemples

- Q9 : Obtenir les noms des buveurs ayant bu du Chablis:

| Solution 1                                                                                                          | Solution 2                                                                                                                                       | Solution 3                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SELECT NOM<br/>FROM BUVEURS B, VINS V, ABUS A<br/>WHERE B.NB=A.NB<br/>AND A.NV=V.NV<br/>AND V.CRU ="Chablis"</b> | <b>SELECT NOM<br/>FROM BUVEURS B<br/>WHERE NB IN<br/><br/>( SELECT NB<br/>FROM ABUS A, VINS V<br/>WHERE A.NV=V.NV AND<br/>V.CRU ="Chablis" )</b> | <b>SELECT NOM<br/>FROM BUVEURS B<br/>WHERE B.NB IN<br/><br/>( SELECT NB FROM ABUS A<br/>WHERE A.NV IN<br/><br/>( SELECT NV FROM VINS V<br/>WHERE V.CRU ="Chablis" )</b> |

# SQL

## Exemples

- Q10 : Obtenir tous les numéros de vins bus par plus de 100 buveurs:

```
SELECT NV
FROM ABUS
GROUP BY NV
HAVING COUNT (*) > 100
```



# SQL

## Exemples

- Q11: Donner les noms des buveurs qui ont bu une quantité de vins plus grande que la quantité bue par au moins un des buveurs d'adresse TOKYO:

```
SELECT B.NOM
FROM BUVEURS B, ABUS A
WHERE B.NB=A.NB
 AND A.QTE > ANY (SELECT A.QTE
 FROM ABUS A, BUVEURS B
 WHERE B.NB=A.NB
 AND B.VILLE="TOKYO")
```

## SOLUTION 2

```
SELECT B.NOM
FROM BUVEURS B, ABUS A
WHERE B.NB=A.NB
 AND A.QTE > (SELECT MIN(A.QTE)
 FROM ABUS A, BUVEURS B
 WHERE B.NB=A.NB
 AND B.VILLE="TOKYO")
```

# SQL

## Exemples

- Q12: Donner les noms des buveurs qui ont bu une quantité de vins plus grande que la quantité bue par n'importe quel buveur d'adresse TOKYO

```
SELECT B.NOM
FROM BUVEURS B, ABUS A
WHERE B.NB=A.NB
 AND A.QTE > ALL (SELECT A.QTE
 FROM ABUS A, BUVEURS B
 WHERE B.NB=A.NB
 AND B.VILLE="TOKYO")
```

### SOLUTION 2

```
SELECT B.NOM
FROM BUVEURS B, ABUS A
WHERE B.NB=A.NB
 AND A.QTE = (SELECT MAX(A.QTE)
 FROM ABUS A, BUVEURS B
 WHERE B.NB=A.NB
 AND B.VILLE="TOKYO")
```

# SQL

## Requête Union

- Q13 :Obtenir les crus de degré supérieur à 13 ou de millésime 1977

```
(SELECT CRU
FROM VINS WHERE DEGRE>13)
```

**UNION**

```
(SELECT CRU
FROM VINS WHERE Millésime=1977)
```

# SQL

## Requête INTERSECT

- Q14 :Obtenir les crus de degré supérieur à 13 et ceux de millésime 1977

```
(SELECT CRU
FROM VINS WHERE DEGRE>13)
```

**INTERSECT**

```
(SELECT CRU
FROM VINS WHERE Millésime=1977)
```

# SQL

## Commande INSERT

Q15 :Insérer le tuple (200, Chablis, 1977, 14) dans la relation VINS:

```
INSERT INTO VINS (NV, CRU, MILLESIME, DEGRE)
VALUES (200, 'Chablis', 1977, 14)
```

# SQL

## Commande UPDATE

Q16 :Mettre la quantité bue à 0 pour tous les buveurs d'adresse TOKYO:

```
UPDATE ABUS
SET QTE=0
WHERE NB IN (SELECT NB
 FROM BUVEURS , ABUS
 WHERE BUVEURS.NB=ABUS.NB
 AND BUVEURS.VILLE= "TOKYO")
```

# SQL

## Commande DELETE

Q17 :Supprimer tous les vins bus par MARTIN:

```
DELETE VINS
WHERE NV IN (SELECT ABUS.NV
 FROM BUVEURS, ABUS
 WHERE ABUS.NB=BUVEURS.NB
 AND BUVEURS.NOM= "MARTIN")
```

## Modification et suppression de table

1) suppression de la table VINS

**DROP TABLE VINS**

4) ajout d'une colonne « prénom » dans la table BUVEURS

**ALTER TABLE BUVEURS**

**ADD ( prenom VARCHAR(15) ) ;**

5) modification du type quantité dans la table ABUS :

**ALTER TABLE ABUS**

**MODIFY ( QTE NUMBER(8,2) ) ;**

6) supprimer la colonne QTE de la table ABUS :

**ALTER TABLE ABUS**

**DROP ( QTE) ;**

7) renommer la table BUVEURS en PERSONNES

**RENAME BUVEURS TO PERSONNES**



# Conclusion

- SQL1 est un standard minimum
- Les versions étendues:
  - SQL2 = Complétude relationnelle
  - SQL3 = Support de l'objet
  - ...
- Sont aujourd'hui intégrées dans les grands SGBD